What is JavaScript?

 Answer:

JavaScript is a **high-level, interpreted programming language** used to make web pages **interactive**.

 It runs in the browser and also on servers (using Node.js).

Difference between var, let, and const

 Answer:

Feature	var	let	const
Scope	Function	Block	Block
Re-declare	□ Yes	□ No	□ No
Re-assign	□ Yes	□ Yes	□ No

Function Scope

 A variable declared inside a function can be used **only inside that function**

BLOCK SCOPE - variable declared inside { } (block) can be used **only inside that block**

What is Hoisting?

 Answer:

Hoisting means **JavaScript moves declarations to the top** of the scope before execution.

"A closure is a function that has access to its outer function's variables even after the outer function has executed." Used in:

- Data hiding
- Private variables

Operator	Meaning
==	Compare value only
===	Compare value + type

What is Event Loop?

 Answer:

JavaScript is **single-threaded**, but it handles async tasks using the **event loop**.

 It manages:

- Call Stack
- Callback Queue

 Simple flow:

1. Code runs in call stack
2. Async tasks go to queue
3. Event loop pushes them back when stack is empty

Is JavaScript Synchronous or Asynchronous?

 JavaScript is SINGLE-THREADED and SYNCHRONOUS by default
 But it can handle **ASYNCHRONOUS operations using Web APIs**

A callback is a function passed into another function to run later.

 Example:

```
function greet(name, callback) {
  console.log("Hello " + name);
  callback();
}

function sayBye() {
  console.log("Bye");
}

greet("Shaba", sayBye);
```

What is Promise?

 Answer:

A Promise represents a value that will be available **in future**.

 States:

- Pending
- Resolved
- Rejected

 Example:

```
let promise = new Promise((resolve, reject) => {
  resolve("Success");
});

promise.then(data => console.log(data));
```

What is async/await?

 Answer:

Used to handle promises in a **clean and readable way**.

 Example:

```
async function getData() {
   let res = await fetch("url");
   let data = await res.json();
   console.log(data);
}
```

 Looks like synchronous code

What is DOM?

 Answer:

DOM (Document Object Model) represents HTML as a **tree structure**.

 JavaScript can:

- Change content
- Change styles
- Handle events

 Example:

```
document.getElementById("demo").innerHTML = "Hello";
```

What is Arrow Function?

 Answer:

Short syntax for writing functions.

 Example:

```
const add = (a, b) => a + b;
```

What is this keyword?

 Answer:

this refers to the **current object**.

 Example:

```
let obj = {
   name: "Shaba",
   greet() {
      console.log(this.name);
   }
}
```

What is Array Methods (IMPORTANT □)

□ **Common methods:**

```
let arr = [1,2,3];

// add
arr.push(4);

// remove
arr.pop();

// loop
arr.map(x => x * 2);

// filter
arr.filter(x => x > 1);
```

undefined	null
Variable declared but no value	Intentional empty value

Scope Type	Where declared	Accessible where
Global	Outside everything	Everywhere
Function	Inside function	Only inside function
Block	Inside {}	Only inside block

Global Scope

□ Variable declared **outside any function or block**

□ Can be accessed **anywhere in the program**

□ **Example:**

```
var a = 10;  // global scope

function test() {
   console.log(a); // □  accessible
}
```

console.log(a); // accessible

Function Scope

 Variable declared **inside a function**

 Can be used **only inside that function**

 Example:

```
function test() {
   var b = 20;
   console.log(b); //  works
}

console.log(b); //  Error
```

 b is **function scoped**

Block Scope

 Variable declared inside { } (block)

 Works with:

- let
- const

 Example:

```
if (true) {
   let c = 30;
   console.log(c); //  works
}

console.log(c); //  Error
```

What is setTimeout?

 Answer:

Runs code after a delay.

```
setTimeout(() => {
   console.log("Hello");
}, 2000);
```

 Key differences:

- Arrow function → no this
- Function → has its own this

What is Spread Operator?

 Answer:

Used to expand elements.

```
let arr1 = [1,2];
let arr2 = [...arr1, 3];
```

1. What is difference between synchronous and asynchronous?

 Answer:

- **Synchronous** → Executes line by line (blocking)
- **Asynchronous** → Executes without waiting (non-blocking)

 Example:

```
console.log("Start");

setTimeout(() => {
   console.log("Async");
}, 1000);

console.log("End");
```

 Output:

```
Start
End
Async
```

---

2. What is call stack?

 Answer:

- A stack that tracks function execution
- Works in **LIFO (Last In First Out)**

 If too many calls → **Stack Overflow**

□ 3. What is callback hell?

□ **Answer:**

Nested callbacks → hard to read & maintain

□ **Example:**

```
login(function() {
   getData(function() {
      processData(function() {
         display();
      });
   });
});
```

□ Solution:

- Promises
- Async/Await

□ 4. What is debouncing?

□ **Answer:**

Limits how often a function runs (used in search bars)

□ **Example:**

```
function debounce(fn, delay) {
   let timer;
   return function() {
      clearTimeout(timer);
      timer = setTimeout(fn, delay);
   }
}
```

□ 5. What is throttling?

□ **Answer:**

Limits function execution **at fixed intervals**

□ Used in:

- Scroll events
- Resize events

□ 6. What is shallow copy vs deep copy?

□ **Answer:**

Shallow Copy	Deep Copy
Copies reference	Copies full value
Changes affect original	No effect

□ **Example:**

```
let obj1 = {a:1};
let obj2 = obj1; // shallow

obj2.a = 10;
console.log(obj1.a); // 10 □
```

□ 7. What is prototype in JavaScript?

□ **Answer:**

Every object has a prototype → used for inheritance

□ 8. What is IIFE?

□ **Answer:**

Immediately Invoked Function Expression

```
(function() {
   console.log("Run immediately");
})();
```

□ 9. What is difference between map and forEach?

□ **Answer:**

map	forEach
returns new array	does not return

map	forEach
used for transformation	used for iteration

□ 10. What is memory leak?

□ **Answer:**

Unused memory not released → slows app

□ Causes:

- Unused variables
- Global variables